Critique DPO

Zihe Liu, Peter Hayes

February 21, 2026

Abstract

This is *very much* a work-in-progress document, which reviews how generative verifiers can be implemented and simple experimental results. Much of this current version are disjointed concepts and messy; my current task is to tidy these reviews up and include the experimental results.

1 Introduction

As large language models (LLMs) become more capable, fine-tuning techniques to align with human intent are increasingly important. A key consideration for aligning these models is how to most effectively use human resources, or model resources in the case where LLMs themselves are used as oracles. Reinforcement learning from Human or AI preferences (RLHF/RLAIF) is the most prominent example of such a technique but is complex and often unstable. Direct Preference Optimization (DPO) has recently been proposed as a simpler and more stable alternative. In this note, we explore how to expand the DPO objective to make better use of feedback data from the human or AI oracles. Specifically, we incorporate the Chain-Of-Thought reasoning of a generative verifier (acting to verify and critique the validity of base models) in the DPO loss term for the base model.

Asking these oracles to provide reasoning for their binary preferences is common practice. In particular, for LLM-as-a-judge style workflows asking the model to provide reasoning (or so-called chain-of-thought) has been shown to actually improve the model predictions, or verifications in the case of generative verifiers. How can we leverage this rationale when fine-tuning our target model to improve performance in a more data-efficient manner?

2 Background

2.1 DPO

For a given prompt x, two completions are sampled from the model $(y_0, y_1) \sim p_\theta(y|x)$ and an oracle judges which they prefer. We denote y_w as the preferred completion and y_l as the other. Typically, the oracle is a human participant; however, the use of LLMs to instead provide feedback has also shown great promise. This process is repeated over N prompts resulting in the pairwise preference dataset $\mathcal{X}_P = \{x, y_w, y_l\}^N$

During the reward modelling phase in RLHF, the preference data is assumed to follow the Bradley-Terry (BT) model. The objective for training the reward model can be framed as a binary classification task with a cross entropy objective:

$$\mathcal{L}_\phi(\mathcal{X}_P) = -\mathbb{E}_{\mathcal{X}_P}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]. \tag{1}$$

During the subsequent RL fine-tuning phase, the trained reward model is then used to score prompt/completion pairs to provide feedback to the language model. The aim is to maximise the following objective w.r.t θ

$$\mathbb{E}_{x\sim\mathcal{X}, y\sim p_\theta(y|x)}[r_\phi(x, y)] - \beta \mathrm{KL}(p_\theta(y|x)||p_{\theta_0}(y|x)). \tag{2}$$

The second term here regularises the fine-tuned model using the KL-divergence to stay close to the state of the LLM before fine-tuning $p_{\theta_0}(y|x)$. The main rationale provided for this is to prevent the model from deviating too far from the distribution on which the reward model is accurate.

In practise the following reward function is used with PPO to update θ.

$$r_{ppo}(x, y) = r_\phi(x, y) - \beta(\log p_\theta(y|x) - \log p_{\theta_0}(y|x)). \tag{3}$$

DPO is derived from the optimal solution to (2); providing a maximum likelihood objective analogous to equation (1), but parameterised by θ instead of ϕ;

$$\mathcal{L}_\theta(\mathcal{X}_P) = -\mathbb{E}_{\mathcal{X}_P}\left[\log \sigma\left(\hat{r}(x, y_w) - \hat{r}(x, y_l)\right)\right], \tag{4}$$

where we have the *implicit reward model*

$$\hat{r}(x, y) = \beta \log \frac{p_\theta(y|x)}{p_{\theta_0}(y|x)}. \tag{5}$$

This formulation has the distinct advantage of not requiring the explicit reward modeling step and avoids the need to perform any reinforcement learning. Furthermore, it has been shown to outperform RLHF in a range of experiments [Rafailov et al., 2024]

In this work we will suggest simple to implement extensions to this objective.

3 Literature Review

We review research using reasoning data into the RL fine-tuning process.

3.1 RLAIF

[Bai et al., 2022] first introduce RL from AI Feedback (RLAIF) to fine-tune models to be helpful and harmless. They use human labels for helpfulness, but LLM-generated labels for ranking harmlessness responses. They experiment with Chain-Of-Thought (CoT) prompting to generate the preferences to rank harmlessness, and find the RL-CAI with CoT seems slightly less helpful but slightly more harmless compared to without CoT. [Lee et al., 2024] compare RLAIF with RLHF on summarization, helpful dialogue generation, and harmless dialogue generation. Experiments show RLAIF achieve comparable, or even better, performance than RLHF even when the labeller model is the same as the base policy model. Again, experiments show CoT created minimal gains in the quality of post-RLAIF model. [Guo et al., 2024] use LLM as a judge to rank preferences for DPO.

3.2 Using CoT for Increased Reasoning Capability

3.2.1 STaR: Self-Taught Reasoner

STaR [Zelikman et al., 2022] first used CoT data for better performance. They used an iterative approach for improving LLM performance through rationale-based fine-tuning. Given a pretrained model M and a dataset $D = \{(x_i, y_i)\}_{i=1}^n$, we start with a small prompt set $\mathcal{P} = \{(x_i^p, r_i^p, y_i^p)\}_{i=1}^P$ where $P \ll D$ that includes examples with their rationales r and answers y. For each example in D, we concatenate the prompt set to form $x_i = (x_1^p, r_1^p, y_1^p, \ldots, x_P^p, r_P^p, y_P^p, x_i)$. The model generates rationales $\hat{r}_i$ and answers $\hat{y}_i$. We filter to keep only rationales that lead to correct answers ($\hat{y}_i = y_i$), fine-tune M on this filtered dataset, and iterate this process. Importantly, each iteration starts from the original pre-trained model M to prevent overfitting. The process continues until performance convergence.

3.2.2 V-STaR: Training Verifiers for Self-Taught Reasoners

Similarly, V-STaR [Hosseini et al., 2024] use an iterative approach to train a generator G^t and verifier V^t. For each problem in the training data, we sample k completions from the generator: $\{\hat{y}_i \sim G(\cdot|\mathbf{x}_i) : i \in \mathcal{D}_{query}\}$. The generated solutions are labeled for correctness (z) using ground truth answers or test cases. We use only correct generated solutions ($z = 1$) to augment the generator training data $\mathcal{D}_{GEN}$. Both correct and incorrect solutions are added to verifier data $\mathcal{D}_{VER}$ with their correctness labels as $(\mathbf{x}_i, \hat{y}_{ij}, z_{ij})$, enabling the verifier to learn from generator mistakes. At iteration t, generator G^t is obtained by fine-tuning the pretrained model on the augmented $\mathcal{D}_{GEN}$. We sample solutions again from

this generator G^t. The verifier V^t is obtained by using $\mathcal{D}_{VER}$ with DPO to fine-tune a base model G_{SFT}, with preference datasets for correct and incorrect answers. This process repeats for up to T iterations to iteratively augment $\mathcal{D}_{GEN}$ and $\mathcal{D}_{VER}$, V^t and G^t.

3.2.3 Generative Verifiers: Reward Modeling as Next-Token Prediction

The prevalent approach for training verifiers in reasoning domains is to fine-tune an LLM as a classifier on a dataset of correct and incorrect solutions generated from a fixed LLM, using binary cross-entropy loss. These verifiers directly assign a numerical score $r_\theta(x, y) \in [0, 1]$ to estimate the probability that a solution y is correct for a problem x. Discriminative LLM-based verifiers do not utilize the text generation capabilities of pretrained LLMs. To address this issue, we [L. Zhang et al., 2024] propose training generative verifiers , which we call GenRM, using standard next-token prediction. GenRM represents solution correctness using the LLM's probability distribution over tokens, instead of predicting a separate numerical score.

Direct Verifier In its simplest form, GenRM predicts whether a solution is correct using a single 'Yes' or 'No' token. This can be done by maximizing $\log p_\theta(\text{'Yes'}|(x, y^+))$ for correct solutions y^+ and $\log p_\theta(\text{'No'}|(x, y^-))$ for incorrect solutions y^-. To do so, we minimize the SFT loss in (2) on the dataset $\mathcal{D}_{\text{Direct}}$ containing problem-solution pairs and a 'Yes' or 'No' verification token:

$$\mathcal{D}_{\text{Direct}} = \{(x, y^+, I), \text{'Yes'}\} \cup \{(x, y^-, I), \text{'No'}\} \tag{6}$$

where $I =$ 'Is the answer correct (Yes/No)?'. At inference, we use the likelihood of the 'Yes' token as the verifier's score for re-ranking solutions.

Unifying Generation and Verification GenRM seamlessly integrates reward modeling with SFT for generating correct solutions. This can be done by simply changing the data mixture in the SFT loss (2) to include both verification and generation tasks. Given a verification dataset $\mathcal{D}_{\text{verify}}$, which can be $\mathcal{D}_{\text{Direct}}$ or $\mathcal{D}_{\text{CoT}}$, GenRM minimizes the loss:

$$\mathcal{L}_{\text{GenRM}}(\theta, \mathcal{D}_{\text{verify}}) = \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{verify}}) + \lambda \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{correct}}) \tag{7}$$

where $\lambda > 0$ controls the mixture ratio between verification and generating correct solutions. This unified training can improve performance via positive transfer between these related tasks.

Chain-of-Thought Verifiers (GenRM-CoT) Since verification often involves nuanced reasoning, generative verifiers can naturally benefit from Chain-of-Thought (CoT). We can generate intermediate reasoning steps before making a decision about correctness, which may identify subtle errors missed by direct verifiers. To train CoT verifiers, we minimize the SFT loss $\mathcal{L}_{\text{GenRM}}$ on dataset $\mathcal{D}_{\text{CoT}}$ containing problem-solution pairs and corresponding verification rationales v_{CoT} appended with a final question I and 'Yes' or 'No' token:

$$\mathcal{D}_{\text{CoT}} = \{(x, y^+, I_{\text{CoT}}), (v_{\text{CoT}}, I, \text{'Yes'})\} \cup \{(x, y^-, I_{\text{CoT}}), (v_{\text{CoT}}, I, \text{'No'})\} \tag{8}$$

where $I_{\text{CoT}} =$ 'Let's verify step by step.'. During inference, we first generate a CoT rationale and then use the probability of 'Yes' for assigning the correctness score.

Data for the CoT verification rationales v_{CoT} is available here

3.3 Using CoT in Preference Optimization

These methods enrich data with reasoning annotations (e.g., explanations, corrections, step-by-step rationales) in the preference optimization setup.

3.3.1 Using Reasoning for Improved LLM Preference Ranking

In the RLAIF process, [Li et al., 2024] instruct the LLM labeller to use the reasoning behind the generation of preference data to rank correctness of the preference pairs. They also use CoT prompting for the LLM labelling.

3.3.2 Self Rewarding Language Models

[W. Yuan et al., 2024] use language models to generate candidate responses, provide its own rewards to these responses, filter for good responses, and do iterative fine-tuning on these good responses. As the model can both improve its generation ability, and act as its own reward model through the same generation mechanism, this means the reward model itself can improve through these iterations, deviating from standard practices where the reward model is fixed.

Self-Instruction Creation We generate a new prompt x_i every iteration using few-shot prompting, sampling prompts from original seed data. We then generate N diverse candidate responses $\{y_i^1, ..., y_i^N\}$ for the given prompt x_i from our model using sampling. Finally, we use the LLM-as-a-Judge ability of our *same model* to evaluate its own candidate responses with scores $r_i^n \in [0, 5]$.

AI Feedback Training After performing the self-instruction creation procedure, we can augment the seed data with additional examples for training, which we refer to as AI Feedback Training (AIFT) data. To do this, we construct preference pairs, which are training data of the form (instruction prompt x_i, winning response y_i^w, losing response y_i^l). To form the winning and losing pair we take the highest and lowest scoring responses from the N evaluated candidate responses, discarding the pair if their scores are the same. These pairs can be used for training with a preference tuning algorithm DPO.

Iterative Training Our overall procedure trains a series of models $M_1, ..., M_T$ where each successive model t uses augmented training data created by the $t-1^{th}$ model. We thus define AIFT(M_t) to mean AI Feedback Training data created using model M_t. This iterative training resembles the procedure used in Pairwise Cringe Optimization and specifically is termed Iterative DPO, introduced in [Xu et al., 2024]; however, an external fixed reward model was used in that work.

3.3.3 Iterative Preference Optimization

[Pang et al., 2024] optimize the preference between competing generated Chain-of-Thought (CoT) candidates by optimizing for winning vs. losing reasoning steps. The use a modified form of the DPO loss with an additional NLL term on the reasoning data. There is a large improvement from 55.6% to 81.6% on GSM8K, and other math benchmarks like MATH and ARC-Challenge.

The authors first construct a dataset of preference pairs, with each pair consisting of a winning (correct) answer and losing (incorrect) answer, along with the LLM's reasoning chain c_i : $D^{\text{pairs}} = \{(c_i^{w_k}, y_i^{w_k}), (c_i^{l_k}, y_i^{l_k})$. Note that due to the task of math reasoning, the reward is binary, either 1 or 0 depending on whether the final answer y_i was correct or not. Given the preference pairs, they then train a new model M_θ that will become the next model M_{i+1}. A sum of the DPO loss and NLL loss over the reasoning chain is used

$$\begin{aligned}\mathcal{L}_{\text{DPO+NLL}} &= \mathcal{L}_{\text{DPO}}(c_i^w, y_i^w, c_i^l, y_i^l|x_i) + \alpha\mathcal{L}_{\text{NLL}}(c_i^w, y_i^w|x_i) \\ &= -\log\sigma\left(\beta\log\frac{M_\theta(c_i^w, y_i^w|x_i)}{M_i(c_i^w, y_i^w|x_i)} - \beta\log\frac{M_\theta(c_i^l, y_i^l|x_i)}{M_i(c_i^l, y_i^l|x_i)}\right) - \alpha\frac{\log M_\theta(c_i^w, y_i^w|x_i)}{|c_i^w| + |y_i^w|}\end{aligned} \tag{9}$$

Here $M(x)$ denotes the probability of sequence x under the model M, and σ is the sigmoid function. We use the previous iteration's model M_i as the reference model in the denominator of the DPO term. Note that the NLL term is normalized by the total response length. The hyperparameter α balances the two loss terms. At the end of this training, we thus obtain our next model M_{i+1} which will be used to generate responses in the next iteration.

Iterative Preference Optimization allows the use of negative examples for feedback, while methods like STaR [Zelikman et al., 2022] or ReST [Gulcehre et al., 2023] only use SFT on the generator and

only include positive examples. [Wu, Lan, et al., 2024] follow using iterative DPO to train the base model with the reasoning chains, but without the NLL loss. [P. Wang et al., 2024] extend this work by employing three approaches to collect the preference pairs for different use cases, afterwards using the same objective in equation 9 to train a better judge model.

3.3.4 AIPO: Improving Training Objective for Iterative Preference Optimization

AIPO [Shen et al., 2024] use a modified version of DPO and SFT:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta(s_\theta - s_{\text{ref}})\right)\right]$$

where

$$s_\theta = \log \frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)}, s_{\text{ref}} = \log \frac{\pi_{\text{ref}}(y_w|x)}{\pi_{\text{ref}}(y_l|x)}.$$

We note that s_{ref} represents the extent to which the reference model model tends to generate the chosen response y_w with a higher probability than the rejected response y_l, i.e., the agreement between the reference model and the reward model. We introduce an additional coefficient α to s_{ref}:

$$\mathcal{L}_{\alpha\text{-DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta(s_\theta - (1+\alpha)s_{\text{ref}})\right)\right]$$

where we set $\alpha > 0$. The final loss is then:

$$\mathcal{L}_{\text{AIPO}} = \mathcal{L}_{\alpha\text{-DPO}} + \lambda \cdot \mathcal{L}_{\text{NLL}}$$

where $\mathcal{L}_{\text{NLL}} = -\frac{1}{|y_w|} \log(\pi_\theta(y_w|x))$

Code available here

3.3.5 Self-Taught Evaluators

[T. Wang et al., 2024] curate a preference dataset (x_i, y_i^w, y_i^l) and use a seed LLM to generate reasoning traces c_i on ranking the outputs. They then iteratively fine-tune the evaluator LLM on the next iteration of reasoning traces (x_i, y_i^w, y_i^l, c_i), to create a better evaluator.

Code and Data Code repository available here, data here, and model weights here. The initial prompts were generated from WildChat, and then model generations and LLM-Judge evaluations are by Llama-3-70B. The dataset contains preference pair generations, as well as the correct rationales for preference pairs ranking (the correct ranking is pre-determined). However, since the task was general QA, we have no simple way to evaluate whether our method RDPO works given that we do use this dataset for RDPO. Training was done using Facebook's internal `fairseq2` library.

3.3.6 Meta-Rewarding Language Models

[Wu, W. Yuan, et al., 2024] uses the model to judge its own judgements and uses that feedback to refine its judgment skills.

3.3.7 Regularized Preference Optimization

[Liu et al., 2024] use a combination of DPO and SFT loss.

Code Repo available here. They use `OpenRLHF` for training and the `UltraFeedback` dataset.

3.4 Using Critique for Reward Modelling

3.4.1 Improving Reward Models with Synthetic Critiques

[Zihuiwen Ye et al., 2024] use externally generated critique of the preference dataset to train them jointly on the reward model. Once we have the critiques c of the preference dataset generated from an external LLM, we augment the training data by enriching them with critiques. We achieve this by concatenating the critiques after each completion to form new preference pairs $(x, y_+ : c_+, y_- : c_-)^2$, which become a critique-augmented training set $\mathcal{D}'$. We train critique RMs on $\mathcal{D}'$ with the loss:

$$\mathcal{L}(\theta, \mathcal{D}') = -\mathbb{E}_{(x,y_+,y_-,c)\sim\mathcal{D}'} \log(\sigma(r_\theta(x, y_+ : c_+) - r_\theta(x, y_- : c_-) - m(r))) \tag{10}$$

We implement our RMs by replacing the final projection layer of a pretrained language model with a linear layer that predicts a logit. This is similar to discriminative verifiers.

3.4.2 Critique-Out-Loud Reward Models

CLoud reward models [Ankner et al., 2024] are trained with a dataset of N examples, $D = \{(x, y^-, y^+, c^-, c^+)_i\}_{i=1}^N$ where we introduce oracle critiques, c^-, c^+, of the rejected and chosen responses y^-, y^+ respectively. The critiques are reasoning traces that provide feedback on the weaknesses of the responses and strategies for improving them.

To train CLoud reward models we: (1) train the base model and LM head to generate critiques via supervised finetuning on the oracle critiques, (2) replace oracle critiques in the dataset with critiques generated by the finetuned model, and (3) train a reward head conditioned on self-generated critiques. We choose to train the reward head on self-generated critiques as to minimizes the distribution shift in the critiques seen by the reward head between training and inference when oracle critiques are not available.

First, we modify L_{RM} to work with CLoud reward models as:

$$L_{RM}(\theta_B, \theta_R, D) = -\mathbb{E}_{(x,y^-,y^+,c^-,c^+)\sim D}[\log(\sigma(r_{\theta_B,\theta_R}(x, y^+, c^+) - r_{\theta_B,\theta_R}(x, y^-, c^-)))]$$

where the reward estimator $r_\theta(x, y, c)$ is now also conditioned on a critique c. Next, we introduce the critique SFT loss, which is the negative log likelihood of the rejected and chosen critiques:

$$L_{SFT}(\theta_B, \theta_{LM}, D) = -\mathbb{E}_{(x,y^-,y^+,c^-,c^+)\sim D}[\sum_{c_t^- \in c^-} \log p(c_t^- | x, y^-, c_{<t}^-; \theta_B, \theta_{LM}) + \sum_{c_t^+ \in c^+} \log p(c_t^+ | x, y^+, c_{<t}^+; \theta_B, \theta_{LM})]$$

where $c_{<t} = c_1, ..., c_{t-1}$ is the prefix of the critique up to the tth token. Finally, we introduce a joint SFT and RM loss as:

$$L_{CLoud}(\theta_B, \theta_{LM}, \theta_R, D) = L_{RM}(\theta_B, \theta_R, D) + \lambda \cdot L_{SFT}(\theta_B, \theta_{LM}, D)$$

where λ is a hyperparameter that weights the contribution of the language modeling loss.

To train CLoud reward models we first train θ_B, θ_{LM} to generate critiques by minimizing L_{SFT} on the oracle critiques in the oracle dataset D and obtain parameters $\tilde{\theta}_B, \tilde{\theta}_{LM}$. We use the finetuned model to modify our dataset with self-generated critiques that replace the oracle critiques. Finally, we obtain our CLoud reward model parameters by training on our self-critique dataset:

$$\theta_B^*, \theta_{LM}^*, \theta_R^* = \arg\min_{\theta_B,\theta_{LM},\theta_R} L_{CLoud}(\theta_B, \theta_{LM}, \theta_R, D_{self})$$

where θ_B, θ_{LM} are initialized to the finetuned parameters $\tilde{\theta}_B, \tilde{\theta}_{LM}$. We train CLoud reward models on the joint loss L_{CLoud} instead of L_{RM} to preserve the critique generation capability of the CLoud reward model and to prevent over-fitting to solely producing reward scores.

Code and Data Code and data available here.

3.4.3 Self-Generated Critiques Boost Reward Modeling for Language Models

Recent work [Zihuiwen Ye et al., 2024] directly incorporates critiques generated from off-the-shelf LLMs for reward modeling, while [Ankner et al., 2024]and [L. Zhang et al., 2024] design a joint training approach for learning to generate the critique as well as rewards simultaneously via knowledge distillation. These methods typically rely on a strong teacher LLM to generate high-quality critiques, which can be costly and inefficient to obtain at scale in practice. Moreover, they cannot be used to improve frontier models when a stronger teacher model does not exist.

Critic-RM [Yu et al., 2025], a framework to allow LLMs to take advantage of self-generated critiques for reward modeling. Critic-RM first generates candidate critiques for each prompt-response pair. Then, a filtering step is conducted to reduce the effect of potentially noisy rationales leading to incorrect predictions, allowing us to augment the preference pairs with additional critiques with the goal of improving the precision of reward modeling. Finally, we implement a joint training scheme to teach the model both high-quality critique generation and accurate reward modeling.

Critique-augmented Reward Model Training We denote z^+, z^- as critiques for chosen and rejected responses y^+, y^- with prompt x, respectively. Then, the overall learning objective $p(y^+ \succ y^-|x)$ can be recast as:

$$\mathcal{L} = \mathbb{E}_{q_\phi(z^+|y^+,x),q_\phi(z^-|y^-,x)}\left[-\log \frac{p(y^+ \succ y^-, z^+, z^-|x)}{q_\phi(z^+|y^+,x)q_\phi(z^-|y^-,x)}\right]$$

$$= \mathbb{E}_{q_\phi(z^+|y^+,x),q_\phi(z^-|y^-,x)}\left[-\log p(y^+ \succ y^-|z^+,z^-,x)\right] + D_{KL}((q_\phi(z^+|y^+,x)||p^*(z^+|y^+,x)) + D_{KL}((q_\phi(z^-|y^-,x)||p^*(z$$

We decompose the reward model training objective into 2 parts:

1) Preference Modeling Loss with Critiques ℓ_r: the reward model r_θ learns to predict the reward for each response conditioned on critiques

$$\ell_r(x, y^+, y^-, z^+, z^-) = -\log p(y^+ \succ y^-, z^+, z^-|x) = -\log p(r_\psi(x, [y^+; z^+]) > r_\psi(x, [y^-; z^-]))$$

2) Critique Generation Loss ℓ_c: the LLM generation g_θ is trained to generate critiques to approximate the oracle distribution $p^*(|y, x)$.

$$\begin{aligned}\ell_c(Z; x, y) &= D_{KL}(p^*(z|y_i, x_i)||q_\phi(z|y_i, x_i)) \\ &= \mathbb{E}_{z\sim p^*(z|y_i,x_i)}[\log p^*(z|y_i, x_i) - \log q_\phi(z|y_i, x_i)] \\ &= -\frac{1}{K}\sum_{z\in Z} \log q_\phi(z|y, x) + \text{const.}\end{aligned}$$

Then, the overall loss for critique generation can be written as $\ell_c(x, y^+, y^-, Z^+, Z^-) = \ell_c(Z^+; x, y^+) + \ell_c(Z^-; x, y^-)$. We then jointly train the reward model r_ψ and critique generation model g_ϕ.

3.4.4 GenRM

In our [Mahan et al., 2024] proposed framework we begin with an LLM π_ϕ acting as a zero-shot judge in an RLAIF setting. That is, given a task x and two responses y_1 and y_2, we directly prompt the model π_ϕ to provide an answer indicator token I indicating a preference over the answers. We consider two variants of our approach:

1. The Generative Reward Model (GenRM) approach prompts the model to act as a classifier directly providing the answer token probabilities for each response $\hat{I} \sim \pi_\phi(I|x, y_1, y_2)$.
2. The CoT-GenRM approach additionally prompts the model to provide intermediate Chain-of-Thought reasoning $\hat{I}, \hat{r} \sim \pi_\phi(I, r|x, y_1, y_2)$ before providing the final answer indicator token.

Our prompts are based on the standard MT-Bench prompt. We use the LLM judge as a prior and further train it to align with the ground-truth dataset judgements. We begin with the preference dataset $\mathcal{D} = \{x^{(i)}, y_1^{(i)}, y_2^{(i)}, I^{(i)}\}_{i=1}^N$, however we consider unranked answers $y_1^{(i)}, y_2^{(i)}$ and the corresponding winning choice $I^{(i)}$.

GenRM (no CoT) To train the GenRM model, we use the standard supervised fine tuning objective

$$\mathcal{L}_{\text{GenRM}}(\pi_\phi) = \mathbb{E}_{(x,y_1,y_2,I)\sim\mathcal{D}}[-\log \pi_\phi(I|x, y_1, y_2)] \tag{11}$$

essentially using the LLM as a classifier trained with next-token prediction.

CoT-GenRM with Rationalization To train the CoT-GenRM we will also consider two settings—bootstrappin intermediate reasoning from ground-truth or, potentially, a stronger rationalization model $r \sim q_\phi(r|x^{(i)}, y_1^{(i)}, y_2^{(i)})$. We can then train the model with maximum likelihood over both the reasoning chain and ranking:

$$\mathcal{L}_{\text{GenRM-Rationalization}}(\pi_\phi) = \mathbb{E}_{(x,y_1,y_2,r,I)\sim\mathcal{D}}[-\log \pi_\phi(I|x, y_1, y_2, r) - \log \pi_\phi(r|x, y_1, y_2)] \tag{12}$$

we refer to this as a post-rationalization approach.

CoT-GenRM-STaR Finally we consider an approach where the model self-bootstraps intermediate reasoning using a STaR approach. We will also consider two loss objectives here—following the filtering strategy described in the above section, we use the standard SFT loss on reasoning chains that yield the correct judgement. We denote models trained with this objective as STaR-SFT. Alternatively, we would like to utilize all the sampled data, including reasoning chains that yield wrong judgments. Similar to the reasoning approach in [Pang et al., 2024] we create a dataset of preference pairs $\mathcal{D} = \{x^{(i)}, y_1^{(i)}, y_2^{(i)}, r_w^{(i)}, I_w^{(i)}, r_l^{(i)}, I_l^{(i)}\}$, where r_w are rationales that lead to correct rankings I_w and r_l are rationales that lead to incorrect rankings I_l. We then use a DPO optimization objective of the form:

$$\mathcal{L}_{\text{GenRM-DPO}}(\pi_\phi) = \mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\phi(I_w, r_w|x, y_1, y_2)}{\pi_{\text{ref}}(I_w, r_w|x, y_1, y_2)} - \beta \log \frac{\pi_\phi(I_l, r_l|x, y_1, y_2)}{\pi_{\text{ref}}(I_l, r_l|x, y_1, y_2)}\right)\right] \tag{13}$$

We denote models trained with this objective as STaR-DPO.

Data for generation of correct and incorrect answers, as well as verification for both answers, are available here.

3.4.5 Generative Judges from Preferences Data

Similar to prior methods, [Ziyi Ye et al., 2024] using an LLM to rank preferences and use the rankings in DPO, and the COT data in SFT.

3.5 Using Critique for Supervised-Fine-Tuning

3.5.1 Critique Fine-Tuning

[Y. Wang et al., 2025] CFT involves training the model to critique a given query-response pair, maximizing the likelihood $P(c|[x; y])$, where c is the annotated critique for a query-response pair $[x; y]$. . CFT yields a consistent 4-10% improvement over SFT on six math benchmarks. Notably, our model Qwen2.5-Math-CFT only requires 1 hour of training on 8xH100 over the 50K examples. It can match or outperform strong competitors like Qwen2.5-Math-Instruct on most benchmarks, which use over 2M samples. Moreover, it can match the performance of SimpleRL, which is a DeepSeek-r1 replication trained with 140x more compute.

3.6 Using Critique in Step-Level Training

[S. Yuan et al., 2025] uses step-level model guided critique and MCTS to iteratively refine the model. [D. Zhang et al., 2024] follow a similar approach. [Xi et al., 2024] create AutoMathCritique — an automated and scalable framework for collecting critique data. This framework consists of three main stages: flawed reasoning path construction, critique generation, and data filtering. [Paul et al., 2024] use critique to refine the model's intermediate steps.

Literature Review Update (2024–2026)

The literature on critique-augmented preference optimisation and generative verifiers has grown rapidly since this document was first drafted. The additions below are organised into five new clusters that extend the existing review in Section **??**.

3.7 Step-Level Preference Optimisation

Standard DPO assigns a single preference signal to a complete response. A growing line of work argues that CoT reasoning requires *per-step* supervision because errors early in a chain propagate forward and cannot be corrected by an outcome-level signal.

3.7.1 Step-DPO

[Lai et al., 2024] construct 10K step-wise preference pairs using self-generated data and apply DPO at the step level, arguing that whole-response comparison provides insufficient localisation of errors in long-chain mathematical reasoning. Applied to Qwen2-72B-Instruct, Step-DPO achieves 70.8% on MATH, surpassing GPT-4 at the time of publication.

3.7.2 Thinking Preference Optimisation (ThinkPO)

yang2025thinkingpreferenceoptimization apply DPO to reasoning depth itself, using short CoT responses as rejected completions and long CoT responses as chosen completions. Without any new training data, ThinkPO improves DeepSeek-R1-Distill-Qwen-7B from 87.4% to 91.2% on MATH500 and increases average output length by 25.9%.

3.7.3 Uncovering the Role of CoT in DPO

gao2025uncoveringimpactchainofthought provide empirical evidence that DPO's success on reasoning tasks is largely *attributable* to the presence of CoT data. Across ten LLMs and the text-to-SQL domain (which lacks CoT), DPO degrades performance in six cases. The analysis shows that CoT in DPO training mitigates reward hacking and improves discriminative calibration—directly motivating the Critique DPO approach of incorporating verifier CoT explicitly into the loss.

3.8 Process Reward Models: From Scalar to Generative

Process Reward Models (PRMs) assign a reward to *each* step of a reasoning chain rather than to the final answer alone. The field has evolved rapidly from human-annotated scalar PRMs toward automated and generative variants.

3.8.1 Let's Verify Step by Step (PRM800K)

lightman2023letsverifystepstep demonstrate that process supervision substantially outperforms outcome supervision on MATH, achieving 78% accuracy with a process-supervised reward model compared to 72.4% for outcome supervision. The paper releases PRM800K, a dataset of 800K step-level human annotations, and remains the canonical reference for step-level feedback in reasoning.

3.8.2 ThinkPRM: Process Reward Models That Think

khalifa2025thinkprmprocessrewardmodels is perhaps the closest existing work to the Critique DPO generative-verifier idea at the process level. ThinkPRM generates a verification chain-of-thought *before* scoring each reasoning step rather than predicting a scalar. Using only 1% of the PRM800K labels, ThinkPRM surpasses discriminative PRMs trained on the full dataset by 8% on GPQA-Diamond, demonstrating that generative CoT verifiers are substantially more data-efficient than scalar counterparts.

3.8.3 GenPRM

zhao2025genprm extend the generative PRM idea by combining natural-language reasoning with executable code verification before assigning a step score. A 7B GenPRM exceeds Qwen2.5-Math-PRM-72B using only 23K training examples, further establishing that generative verifier reasoning provides a strong inductive bias for process supervision.

3.8.4 Lessons in Developing PRMs

zhang2025lessonsdevelopingprocessreward provide an important empirical caveat: Monte-Carlo estimation methods (e.g. Math-Shepherd) yield inferior PRMs compared to LLM-as-judge and human annotation. PRMs trained with MC estimation tend to focus assessment on final-answer steps rather than intermediate steps, suggesting that critique-style verification (as in GenRM-CoT and Critique DPO) may better capture the true step-level quality signal.

3.9 Training Critique and Self-Correction Models

A second growing cluster trains models specifically to *generate* critiques or corrections, rather than using externally provided ones.

3.9.1 CriticGPT

mcaleese2024llmcriticshelpllm train an LLM critique model via RLHF on human-annotated errors injected into ChatGPT outputs. CriticGPT-trained critics are preferred over unassisted human critics in 63% of cases and catch ∼85% of bugs versus ∼25% for humans alone. The paper validates the scalable-oversight motivation for critique-based training and provides a practical data construction recipe.

3.9.2 CTRL: Teaching Critique via RL

xie2025teachinglanguagemodelscritique (CTRL) train a critic model with RL to maximise correction performance for a fixed generator, without requiring human supervision. The approach produces critiques that serve as accurate generative reward signals and enables test-time scaling through iterative critique–revision cycles, with up to 106% relative improvement on challenging code-generation benchmarks.

3.9.3 SuperCorrect

yang2024supercorrect introduce a two-stage framework: (1) hierarchical thought template distillation from a large teacher model, and (2) cross-model collaborative DPO training on teacher-guided error-correction traces. SuperCorrect-7B surpasses DeepSeekMath-7B by 7.8%/5.3% on MATH/GSM8K. The error-correction trace is a distilled form of the generative verifier output that Critique DPO proposes to include in the DPO loss.

3.9.4 Critique-GRPO

zhang2025critiquegrpoadvancingllm is the closest existing paper to Critique DPO's core proposal: it integrates natural-language critiques *and* numerical rewards jointly in a GRPO-based RL framework. The method enables LLMs to learn from both initial responses and critique-guided self-refinements simultaneously, achieving +4.4% on Qwen2.5-7B-Base and +16.7% pass@1 on AIME 2024. The key difference from Critique DPO is the use of GRPO rather than DPO, and the online RL rather than offline preference optimisation setting.

3.9.5 RL Tango: Joint Generator–Verifier Training

zha2025rltangoreinforcinggenerator simultaneously train both a policy generator and a generative process-level verifier using interleaved RL, with the verifier rewarded solely on outcome correctness without explicit process labels. The co-evolving pair achieves best-in-class performance on competition mathemat-

ics while the verifier leads ProcessBench, providing practical evidence that the joint generator–verifier training loop in Critique DPO can work without pre-labelled critique data.

3.10 Generative Reward Models and Thinking LLM-as-Judge

A third cluster directly extends the *generative verifier* idea of [L. Zhang et al., 2024] toward preference ranking and reward modelling at scale.

3.10.1 Prometheus 2 and RewardBench

kim2024prometheus2 release Prometheus 2, an open-source evaluator LLM trained on a combined SFT + preference dataset that supports both direct assessment and pairwise ranking using custom scoring rubrics. It achieves the highest correlation with human evaluators among open-source alternatives, confirming the utility of critique-style reasoning in evaluation.

lambert2024rewardbench introduce RewardBench, the standard benchmark for evaluating reward models across chat, reasoning, and safety domains. The benchmark's *reasoning* category is the most relevant for evaluating Critique DPO-trained models, as it tests preference models specifically on mathematical and coding tasks.

3.10.2 EvalPlanner (Thinking LLM-as-a-Judge)

saha2025evalplanner train an evaluator LLM to generate an *evaluation plan*, execute it through reasoning, and then deliver a final judgement. A self-training loop with synthetic preference pairs iteratively refines the evaluation plans. EvalPlanner achieves 93.9 on RewardBench—state-of-the-art among generative reward models at the time of publication—demonstrating that planning-then-judging substantially outperforms direct preference prediction.

3.10.3 J1: Thinking LLM-as-a-Judge via RL

whitehouse2025j1incentivizingthinkingllmasajudge apply GRPO to teach judges to *think before judging* by converting judgement tasks into formats with verifiable rewards and adding consistency-based reward bonuses to reduce positional bias. J1-Qwen-32B outperforms o1-mini, o3, and DeepSeek-R1-671B on several judge benchmarks, trained only on synthetic data. The training methodology is essentially Critique DPO's core idea applied to a standalone judge model via RL.

3.10.4 RM-R1: Reward Modelling as Reasoning

chen2025rmr1rewardmodelingreasoning train Reasoning Reward Models (ReasRMs) in two stages: (1) chain-of-rubrics distillation from a large model, and (2) RL with verifiable rewards. For chat tasks the model generates evaluation rubrics and justifications; for reasoning tasks it solves the problem itself before judging. RM-R1 outperforms INF-ORM-Llama3.1-70B and GPT-4o on reward model benchmarks by up to 4.9%, establishing that structured reasoning before scalar prediction is the right paradigm for reward modelling.

3.10.5 DeepSeek-GRM and Self-Principled Critique Tuning (SPCT)

liu2025deepseekgrm introduce SPCT, an online RL method where the reward model adaptively generates *principles and critiques* for each input before scoring, trained via RL to maximise downstream alignment quality. Inference-time scaling is achieved by sampling multiple critique chains in parallel, guided by a meta-reward model. SPCT achieves better performance than training-time scaling and avoids systematic biases present in traditional reward models. This can be viewed as a production-scale implementation of the Critique DPO reward-modelling idea, with the critic output used directly as the reward signal.

References

Rafailov, Rafael et al. (2024). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. arXiv: 2305.18290 [cs.LG]. URL: https://arxiv.org/abs/2305.18290.

Bai, Yuntao et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv: 2212.08073 [cs.CL]. URL: https://arxiv.org/abs/2212.08073.

Lee, Harrison et al. (2024). *RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback*. arXiv: 2309.00267 [cs.CL]. URL: https://arxiv.org/abs/2309.00267.

Guo, Shangmin et al. (2024). *Direct Language Model Alignment from Online AI Feedback*. arXiv: 2402.04792 [cs.AI]. URL: https://arxiv.org/abs/2402.04792.

Zelikman, Eric et al. (2022). *STaR: Bootstrapping Reasoning With Reasoning*. arXiv: 2203.14465 [cs.LG]. URL: https://arxiv.org/abs/2203.14465.

Hosseini, Arian et al. (2024). *V-STaR: Training Verifiers for Self-Taught Reasoners*. arXiv: 2402.06457 [cs.LG]. URL: https://arxiv.org/abs/2402.06457.

Zhang, Lunjun et al. (2024). *Generative Verifiers: Reward Modeling as Next-Token Prediction*. arXiv: 2408.15240 [cs.LG]. URL: https://arxiv.org/abs/2408.15240.

Li, Ang et al. (2024). *HRLAIF: Improvements in Helpfulness and Harmlessness in Open-domain Reinforcement Learning From AI Feedback*. arXiv: 2403.08309 [cs.LG]. URL: https://arxiv.org/abs/2403.08309.

Yuan, Weizhe et al. (2024). *Self-Rewarding Language Models*. arXiv: 2401.10020 [cs.CL]. URL: https://arxiv.org/abs/2401.10020.

Xu, Jing et al. (2024). *Some things are more CRINGE than others: Iterative Preference Optimization with the Pairwise Cringe Loss*. arXiv: 2312.16682 [cs.CL]. URL: https://arxiv.org/abs/2312.16682.

Pang, Richard Yuanzhe et al. (2024). *Iterative Reasoning Preference Optimization*. arXiv: 2404.19733 [cs.CL]. URL: https://arxiv.org/abs/2404.19733.

Gulcehre, Caglar et al. (2023). *Reinforced Self-Training (ReST) for Language Modeling*. arXiv: 2308.08998 [cs.CL]. URL: https://arxiv.org/abs/2308.08998.

Wu, Tianhao, Janice Lan, et al. (2024). *Thinking LLMs: General Instruction Following with Thought Generation*. arXiv: 2410.10630 [cs.CL]. URL: https://arxiv.org/abs/2410.10630.

Wang, Peifeng et al. (2024). *Direct Judgement Preference Optimization*. arXiv: 2409.14664 [cs.CL]. URL: https://arxiv.org/abs/2409.14664.

Shen, Yaojie et al. (2024). *AIPO: Improving Training Objective for Iterative Preference Optimization*. arXiv: 2409.08845 [cs.CL]. URL: https://arxiv.org/abs/2409.08845.

Wang, Tianlu et al. (2024). *Self-Taught Evaluators*. arXiv: 2408.02666 [cs.CL]. URL: https://arxiv.org/abs/2408.02666.

Wu, Tianhao, Weizhe Yuan, et al. (2024). *Meta-Rewarding Language Models: Self-Improving Alignment with LLM-as-a-Meta-Judge*. arXiv: 2407.19594 [cs.CL]. URL: https://arxiv.org/abs/2407.19594.

Liu, Zhihan et al. (2024). *Provably Mitigating Overoptimization in RLHF: Your SFT Loss is Implicitly an Adversarial Regularizer*. arXiv: 2405.16436 [cs.LG]. URL: https://arxiv.org/abs/2405.16436.

Ye, Zihuiwen et al. (2024). *Improving Reward Models with Synthetic Critiques*. arXiv: 2405.20850 [cs.CL]. URL: https://arxiv.org/abs/2405.20850.

Ankner, Zachary et al. (2024). *Critique-out-Loud Reward Models*. arXiv: 2408.11791 [cs.LG]. URL: https://arxiv.org/abs/2408.11791.

Yu, Yue et al. (2025). *Self-Generated Critiques Boost Reward Modeling for Language Models*. arXiv: 2411.16646 [cs.CL]. URL: https://arxiv.org/abs/2411.16646.

Mahan, Dakota et al. (2024). *Generative Reward Models*. arXiv: 2410.12832 [cs.LG]. URL: https://arxiv.org/abs/2410.12832.

Ye, Ziyi et al. (2024). *Beyond Scalar Reward Model: Learning Generative Judge from Preference Data*. arXiv: 2410.03742 [cs.CL]. URL: https://arxiv.org/abs/2410.03742.

Wang, Yubo, Xiang Yue, and Wenhu Chen (2025). *Critique Fine-Tuning: Learning to Critique is More Effective than Learning to Imitate*. arXiv: 2501.17703 [cs.CL]. URL: https://arxiv.org/abs/2501.17703.

Yuan, Siyu et al. (2025). *Agent-R: Training Language Model Agents to Reflect via Iterative Self-Training*. arXiv: 2501.11425 [cs.AI]. URL: https://arxiv.org/abs/2501.11425.

Zhang, Di et al. (2024). *LLaMA-Berry: Pairwise Optimization for O1-like Olympiad-Level Mathematical Reasoning*. arXiv: 2410.02884 [cs.AI]. URL: https://arxiv.org/abs/2410.02884.

Xi, Zhiheng et al. (2024). *Enhancing LLM Reasoning via Critique Models with Test-Time and Training-Time Supervision*. arXiv: 2411.16579 [cs.CL]. URL: https://arxiv.org/abs/2411.16579.

Paul, Debjit et al. (2024). *REFINER: Reasoning Feedback on Intermediate Representations*. arXiv: 2304.01904 [cs.CL]. URL: https://arxiv.org/abs/2304.01904.

Lai, Xin et al. (2024). *Step-DPO: Step-wise Preference Optimization for Long-chain Reasoning of LLMs*. arXiv: 2406.18629 [cs.LG]. URL: https://arxiv.org/abs/2406.18629.